MVC PROJECT

MT1008: MULTIVARIABLE CALCULUS

Solar Energy Optimization

Multivariable Modeling of Photovoltaic Panel Efficiency

SUBMITTED BY

Ahmad Ali
Group Leader | 25i-0817

Liaba Naveed
25i-0818

Muhammad Osaid Zahid
25i-0524

Submitted To:
Ms. Iqra Arshad
Instructor, Multivariable Calculus

Contents

1 Introduction **2**
- 1.1 Task Description . . . 2
- 1.2 Objectives . . . 2
- 1.3 Methodology . . . 3

2 Python Codes and Output Analysis **4**
- 2.1 Code (Written from Visual Studio Code and google Colab) . . . 5
- 2.2 Output Show . . . 8
- 2.3 Output Graph . . . 9

3 Explaining the Code **10**
- 3.1 Method We Use to Model the Function . . . 10
  - 3.1.1 Method of Regression (Important for Finding the Modelled Function) . . . 10
  - 3.1.2 Regression Flowchart . . . 11
  - 3.1.3 Collect data, identify inputs and output . . . 11
  - 3.1.4 Build design matrix X and target y . . . 11
  - 3.1.5 Define MLR model (assume linear equation) . . . 12
  - 3.1.6 Define loss function . . . 13
  - 3.1.7 Task-by-Task Code Explanation: The `sklearn` Advantage . . . 13
- 3.2 Task Evaluation . . . 16

4 Part II : Handwritten Solutions **19**
- 4.1 Task 2.1: Handwritten Solution . . . 19

5 Docx Written Solution Covering Tasks 1 to 6 **33**
- 5.1 Task 1: Maximum and Minimum Power Conditions . . . 33
- 5.2 Task 2: Gradient Vector $\nabla P(G, T, \theta)$ . . . 34
- 5.3 Task 3: Steepest Descent . . . 34
- 5.4 Task 4: Directional Derivative . . . 35
- 5.5 Task 5: Compute Triple Integral Over a Region . . . 36
- 5.6 Task 6: Performing Location . . . 37

6 Analysis of Results and Conclusion **38**
- 6.1 Analysis of Results . . . 38
- 6.2 Conclusion . . . 39
- 6.3 What We Learned from This Project . . . 40

1. Introduction

Project Overview. The project titled *"Solar Energy Optimization: Multivariable Modeling of Photovoltaic Panel Efficiency"* applies multivariable calculus concepts to a realistic photovoltaic (PV) power prediction problem. Using a real-world inspired 30-day dataset, a regression model is constructed and then analysed through gradient computation, directional derivatives, and triple integration.

1.1 Task Description

The primary task involves developing a mathematical model $P(G, T, \theta)$ that predicts photovoltaic power output (measured in Watts) based on three independent variables:

- **Solar Irradiance** G ranging from 200 to 1000 W/m^2
- **Panel Temperature** T ranging from 15 to 45 °C
- **Angle of Incidence** θ ranging from 0° to 60°

The next step is to use optimisation methods including gradient computation, directional derivatives, and triple integration in order to get useful information for designing solar energy systems.

1.2 Objectives

The analysis targets the following specific outcomes:

- ✔ Construction of a polynomial function via regression that captures temperature losses and angular dependencies
- ✔ Identification of maximum and minimum power conditions with variable sensitivity analysis showing in which direction the power output increases fastest or vice versa
- ✔ Computation of gradient vector $\nabla P(G, T, \theta)$ to determine steepest ascent and descent directions
- ✔ Evaluation of directional derivatives for weather scenario predictions

- ✔ Triple integration over the operational domain to estimate total daily energy potential
- ✔ Location-specific performance comparison for optimal site selection

1.3 Methodology

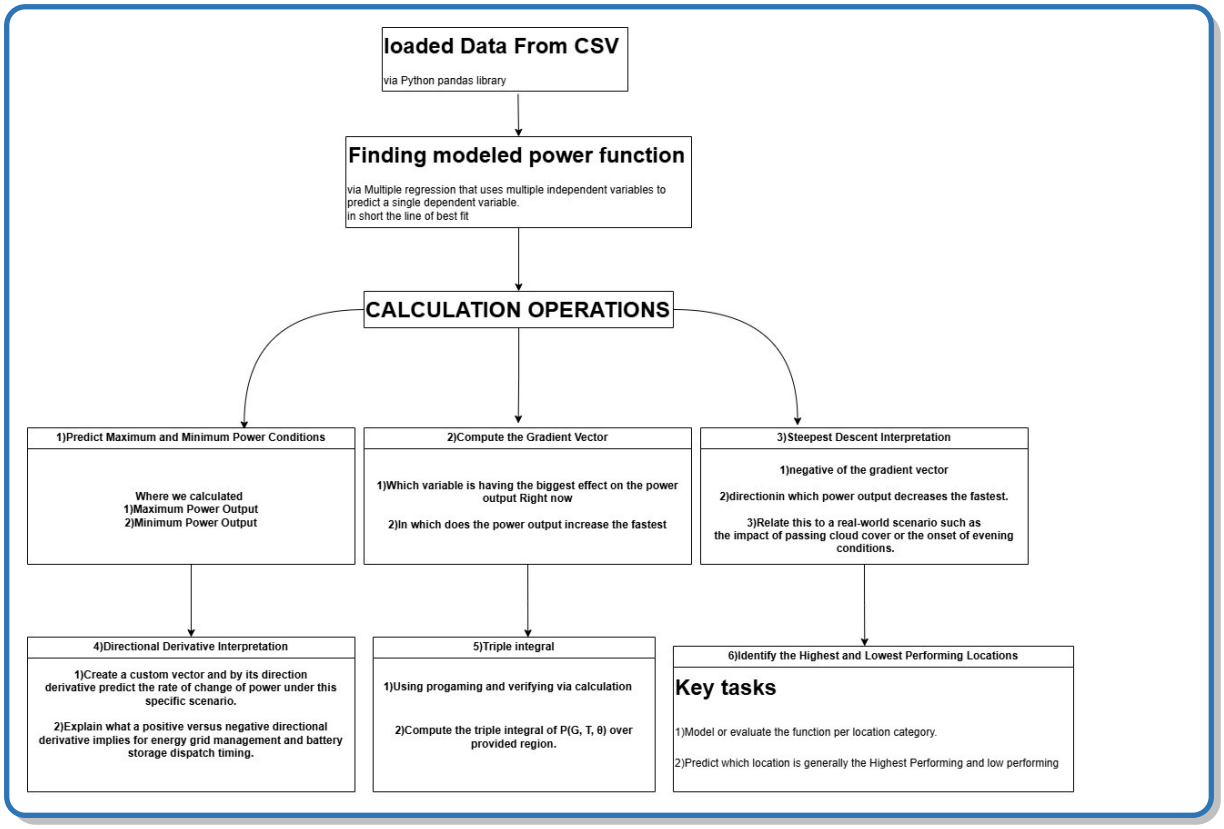


Figure 1.1: Flowchart illustrating the overall methodology adopted for the project

2. Python Codes and Output Analysis

This section presents our Python code and its output, developed and executed using Visual Studio Code and Google Colab. The image below shows a screenshot taken from Google Colab.

The images are on the next pages, kindly view from there!

2.1 Code (Written from Visual Studio Code and google Colab)

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from itertools import product
from scipy import integrate
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import warnings
warnings.filterwarnings('ignore')

#Read data from csv using pandas
df = pd.read_excel("Dataset_30days.xlsx")
df.head()

#define data fragment
X = df[['G_Irradiance', 'Temperature', 'Theta_deg']]
y = df['Power']

#SPlit 100/400 for testing and training
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=13)

#Define regression
lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)
residuals = y_test - y_pred
SSR = np.sum(residuals**2)
print("\nMODEL ERROR ANALYSIS")
print(f"SSR = {SSR:.4f}")
# Coefficients
b0 = lr.intercept_
bG, bT, bθ = lr.coef_

print("So the model is:")
print(f"P = {b0:.4f} + {bG:.4f}·G + {bT:.4f}·T + {bθ:.4f}·θ")
print(f"R**2 = {r2_score(y_test, y_pred):.4f}")
```

Figure 2.1: Python Source Code (Part 1)

```python
#   TASK 1
#Since its a linear model, min max will be at the corners so we extract them
corners = list(product([200, 1000], [15, 45], [0, 60]))
powers  = [lr.predict([[G, T, th]])[0] for G, T, th in corners]

max_i = np.argmax(powers)
min_i = np.argmin(powers)

print("\nTASK 1: MAX AND MIN POWER ")
print("MAX:", round(powers[max_i], 2), "W  at  G=", corners[max_i][0], "T=", corners[max_i][1], "θ=", corners[max_i][2])
print("MIN:", round(powers[min_i], 2), "W  at  G=", corners[min_i][0], "T=", corners[min_i][1], "θ=", corners[min_i][2])

#Strongest variable is which had the largest absolute coefficient
names = ['G_Irradiance', 'Temperature', 'Theta_deg']
print("Strongest influence:", names[np.argmax(np.abs(lr.coef_))])


#   TASK 2
#Gradients would simply be the coefficients cause its a linear function
G0, T0, θ0 = df['G_Irradiance'].mean(), df['Temperature'].mean(), df['Theta_deg'].mean()
grad = np.array([bG, bT, bθ])

print("\nTASK 2: GRADIENT")
print(f"Operating point: G={G0:.1f}, T={T0:.1f}, θ={θ0:.1f}")
print(f"∇P = [{bG:.4f}, {bT:.4f}, {bθ:.4f}]")
print("Largest component:", names[np.argmax(np.abs(grad))])

#Unit vector would be the direction of steepest increase
unit = grad / np.linalg.norm(grad)
print(f"Fastest increase direction (unit vector): [{unit[0]:.4f}, {unit[1]:.4f}, {unit[2]:.4f}]")


#     TASK 3
#Negative gradient is the direction of fastest power decrease
neg_grad = -grad

print("\nTASK 3: STEEPEST DESCENT ")
print(f"-∇P = [{neg_grad[0]:.4f}, {neg_grad[1]:.4f}, {neg_grad[2]:.4f}]")
print("Power drops fastest when G decreases (cloud cover) and θ increases")
```

Figure 2.2: Python Source Code (Part 2)

```python
#   TASK 4
#We take a scenario where irradiance drops, temperature is steady and angle slightly increases
d = np.array([-1.0, 0.0, 0.5])
d_unit = d / np.linalg.norm(d)
dir_deriv = np.dot(grad, d_unit)

print("\n TASK 4: DIRECTIONAL DERIVATIVE")
print(f"Direction (ΔG=-1, ΔT=0, Δθ=+0.5): {d_unit.round(4)}")
print(f"D_u P = {dir_deriv:.4f} W per unit step")
print("Negative meaning power is decreasing ")


#   TASK 5
#Integrate P(G,T,θ) over full operating envelope
G1, G2   = 200, 1000
T1, T2   = 15,  45
th1, th2 = 0,   60

def P(th, T, G):
    return b0 + bG*G + bT*T + bθ*th

result, _ = integrate.tplquad(P, G1, G2, T1, T2, th1, th2)
avg_power = result / ((G2-G1) * (T2-T1) * (th2-th1))

print("\nTASK 5: TRIPLE INTEGRAL")
print(f"∭ P dG dT dθ = {result:.2f}")
print(f"Average power over operating envelope = {avg_power:.2f} W")


#    TASK 6

locations = {
    "Desert"     : [950, 42, 10],
    "Coastal"    : [700, 28, 25],
    "High-Alt."  : [900, 18,  5],
    "Urban"      : [550, 35, 40],
}

print("\nTASK 6: LOCATION PERFORMANCE")
loc_powers = {name: lr.predict([vals])[0] for name, vals in locations.items()}
for name, p in loc_powers.items():
```

Figure 2.3: Python Source Code (Part 3)

```python
        print(f"  {name:<12}: {p:.2f} W")
    print("Best :", max(loc_powers, key=loc_powers.get))
    print("Worst:", min(loc_powers, key=loc_powers.get))

    #  GRAPHS
    # Actual vs Predicted
    plt.scatter(y_test, y_pred, alpha=0.6)
    plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
    plt.xlabel('Actual Power (W)')
    plt.ylabel('Predicted Power (W)')
    plt.title('Actual vs Predicted')
    plt.show()

    # Irradiance vs Power
    plt.scatter(df['G_Irradiance'], df['Power'], alpha=0.5)
    plt.xlabel('G_Irradiance (W/m²)')
    plt.ylabel('Power (W)')
    plt.title('Irradiance vs Power')
    plt.show()
```

Figure 2.4: Python Source Code (Part 4)

2.2 Output Show

This is the terminal output of the Python code that we have done. It shows results for all six tasks including model error analysis, max and min power, gradient, steepest descent, directional derivative, triple integral, and location performance.

```
MODEL ERROR ANALYSIS
SSR = 171510.0427
R**2 = 0.9617

The predicted model is:
P = 218.3116 + 0.8075·G + -2.4235·T + -4.9185·θ

TASK 1: MAX AND MIN POWER
MAX: 989.44 W  at  G= 1000 T= 15 θ= 0
MIN: -24.36 W  at  G= 200 T= 45 θ= 60
Strongest influence: Theta_deg

TASK 2: GRADIENT
Operating point: G=592.0, T=30.1, θ=30.1
∇P = [0.8075, -2.4235, -4.9185]
Largest component: Theta_deg
Fastest increase direction (unit vector): [0.1457, -0.4373, -0.8874]

TASK 3: STEEPEST DESCENT
-∇P = [-0.8075, 2.4235, 4.9185]
Power drops fastest when G decreases (cloud cover) and θ increases

 TASK 4: DIRECTIONAL DERIVATIVE
Direction (ΔG=-1, ΔT=0, Δθ=+0.5): [-0.8944  0.      0.4472]
D_u P = -2.9219 W per unit step
Negative meaning power is decreasing

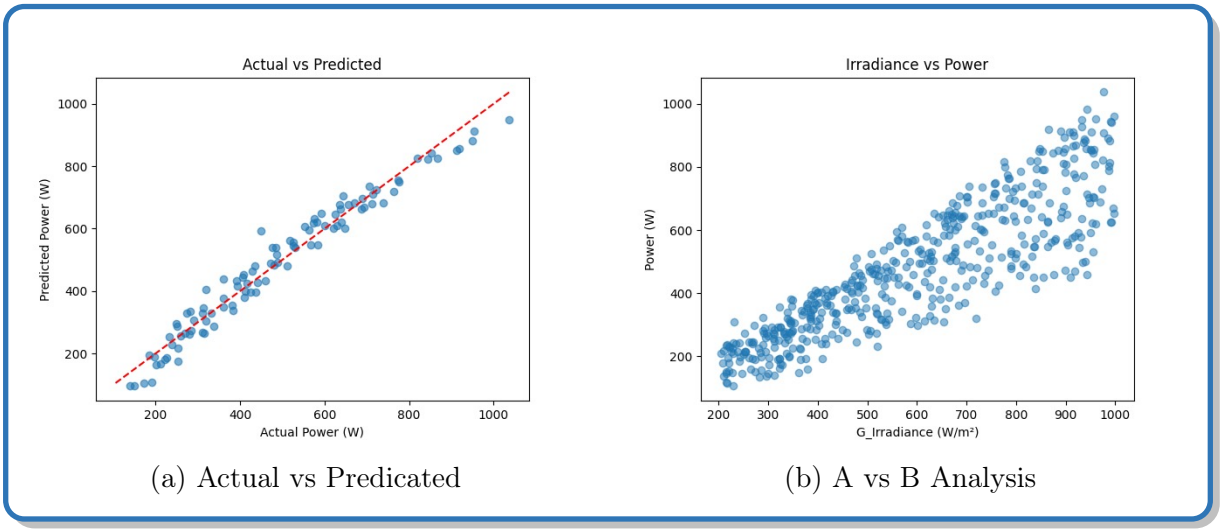
TASK 5: TRIPLE INTEGRAL
∭ P dG dT dθ = 694855713.74
Average power over operating envelope = 482.54 W

TASK 6: LOCATION PERFORMANCE
  Desert      : 834.44 W
  Coastal     : 592.73 W
  High-Alt.   : 876.83 W
  Urban       : 380.86 W
Best : High-Alt.
Worst: Urban
```

Figure 2.5: Terminal output of the Python script showing all task results

2.3 Output Graph

The graph below was generated by the Python code. It shows the relationship between Irradiance and Power output across the 30-day dataset, confirming the strong positive linear relationship.



(a) Actual vs Predicated (b) A vs B Analysis

Figure 2.6: Irradiance vs Power

3. Explaining the Code

3.1 Method We Use to Model the Function

3.1.1 Method of Regression (Important for Finding the Modelled Function)

As it is mentioned in the project brief, and since it was mentioned that:

> Using this dataset, students are required to model a function:
>
> $$P(G, T, \theta)$$
>
> which represents the **power output** of a solar panel. This can be achieved using regression techniques (e.g., `sklearn`) or other suitable methods.

Figure 3.1: From project Task 1

And since it was mentioned that

> *"You are free to adopt any technique and solution methodology to solve this problem."*

Therefore, we have used the **regression technique**. It is basically a technique of finding the approximate prediction based on the data set. For a regression technique we did the following techniques which you can see in our code.

We apply multi-regression since 3 independent variables come. It works in the following way as shown in the flowchart. Each step covers the important part. Kindly see our flowchart for a better explanation of the Regression method used to model the main equation.

The general form of the multi-linear model is:

$$P(G, T, \theta) = b_0 + b_G G + b_T T + b_\theta \theta$$

where b_0 is the intercept and b_G, b_T, b_θ are the learned coefficients.

3.1.2 Regression Flowchart

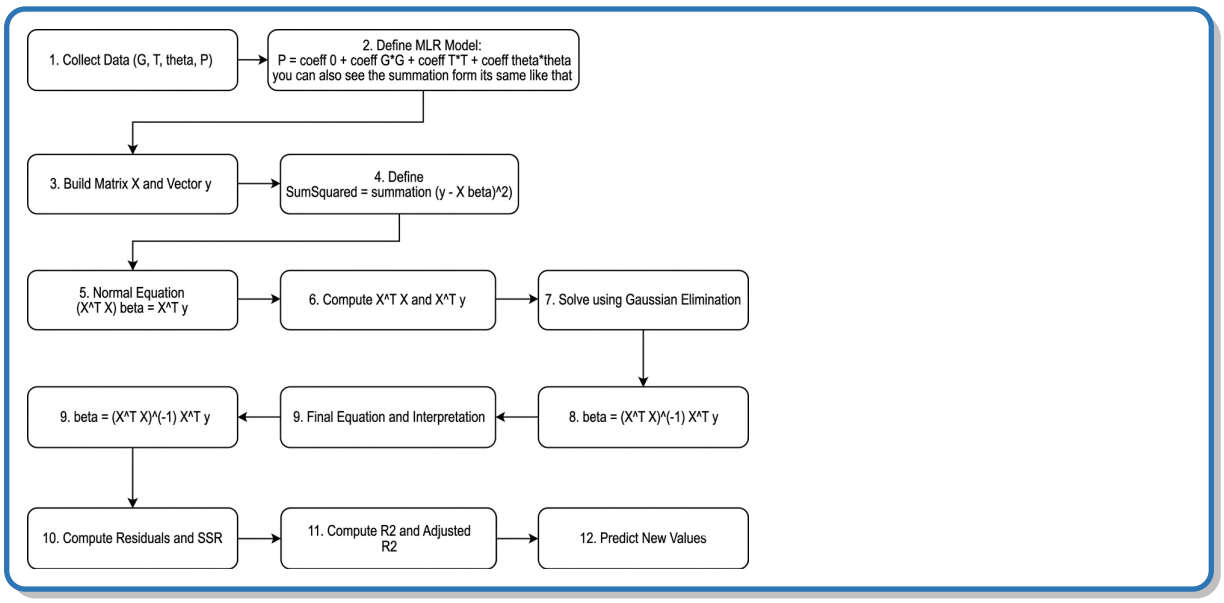


Figure 3.2: Regression pipeline flowchart from data collection to prediction (12 steps)

3.1.3 Collect data, identify inputs and output

Data Fragment Loading part

```
df = pd.read_excel("Dataset_30days.xlsx")
```

Here pd is a name use for panda where panda is a python library used for
Read the .csv,.txt and other document files
This will read the entire .xlsx file into a single data frame. In this context, the data frame acts as an Excel sheet holding the information.

3.1.4 Build design matrix X and target y

Data Fragment Loading part

```
 X = df[['G_Irradiance', 'Temperature', 'Theta_deg']]
y = df['Power']
```

In our code, we build the following matrices:

$$\text{Prediction output matrix } X = \begin{bmatrix} 1 & G_1 & T_1 & \theta_1 \\ 1 & G_2 & T_2 & \theta_2 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & G_n & T_n & \theta_n \end{bmatrix}$$

$$\text{Actual output matrix } y = \begin{bmatrix} P_1 \\ P_2 \\ \vdots \\ P_n \end{bmatrix}^T$$

The same matrices are constructed in our code using the implementation shown above.

3.1.5 Define MLR model (assume linear equation)

Data Fragment Loading part

```
lr = LinearRegression()
```

Here, the general form is:

$$\sum_{i=1}^{n} (a_i \cdot b_i) + b_0$$

Instead of a manual summation loop, we employ `LinearRegression()` to perform the calculation. This allows the code to benefit from vectorized operations, ensuring optimal performance and efficiency.

3.1.6 Define loss function

Loss Optimization

```
lr.fit(X_train, y_train)
```

Whenever a line is drawn through data points, some deviation is inevitable. The difference between the actual output y_i and the predicted value $\hat{y}_i$ is referred to as the residual. As shown below

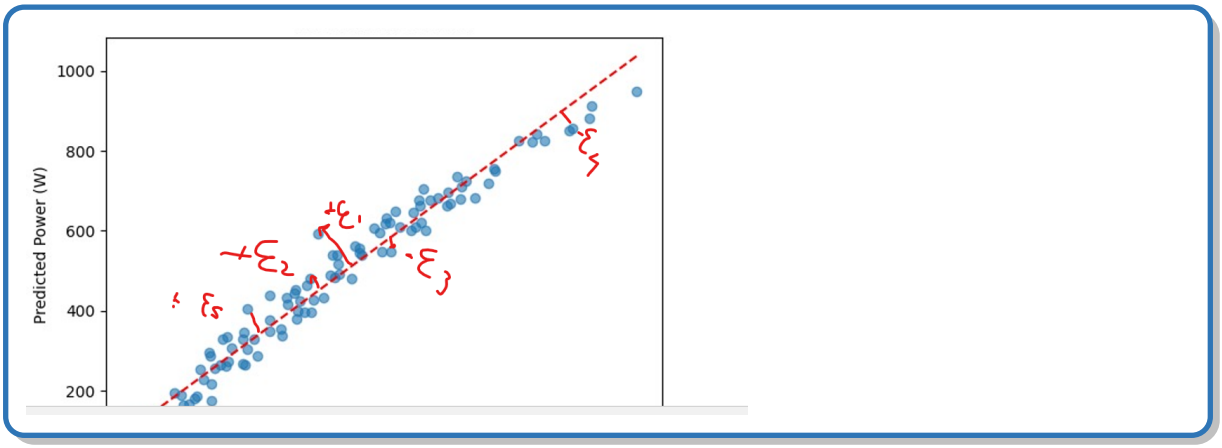


Figure 3.3: Shows loss distance from separate point

If we simply sum the raw residuals, positive and negative errors may cancel each other out, potentially resulting in a total error of zero even when the model is inaccurate. To ensure that every error contributes positively to our total loss, we square each residual before summing them. This yields the Sum of Squared Residuals (SSR):

$$SSR = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{3.1}$$

In our code, we implement this optimization via the `LinearRegression` object: By calling `lr.fit()`, the model automatically computes the optimal coefficients by minimizing the SSR. This approach leverages vectorized internal operations, ensuring computational efficiency and numerical stability compared to manual iterative summation.

3.1.7 Task-by-Task Code Explanation: The `sklearn` Advantage

To bridge the gap between theoretical computation and practical implementation, we utilize the `scikit-learn` library. While manual steps (Steps 4 through 8 in our workflow) are essential for understanding the mathematical foundation of regression, `LinearRegression()` automates these processes for improved performance.

- **Normal Equation Setup ($X^T X\boldsymbol{\beta} = X^T y$):** Instead of manually building matrices and computing the product $X^T X$, the `.fit()` method handles this internally. It solves the system using numerically stable techniques like QR decomposition.

- **Solving via Gaussian Elimination:** While we understand the logic of Gaussian elimination for solving linear systems, `sklearn` employs highly optimized numerical solvers designed for speed and precision, which outperform manual elimination on large datasets.

- **Coefficient Extraction ($\boldsymbol{\beta}$ Values):** After fitting the model, the intercept and coefficients are extracted directly from the object:

Coefficient Extraction

```
b0 = lr.intercept_
bG, bT, bTheta = lr.coef_
```

These values represent the $\boldsymbol{\beta}$ coefficients for our model. The relationship is defined as:

$$P = \beta_0 + \beta_1 G + \beta_2 T + \beta_3 \theta + \epsilon \tag{3.2}$$

where β_0 is the intercept, $\beta_1, \beta_2, \beta_3$ are the coefficients for the independent variables (G, T, θ), and ϵ represents the residual error, which we analyze in Step 10.

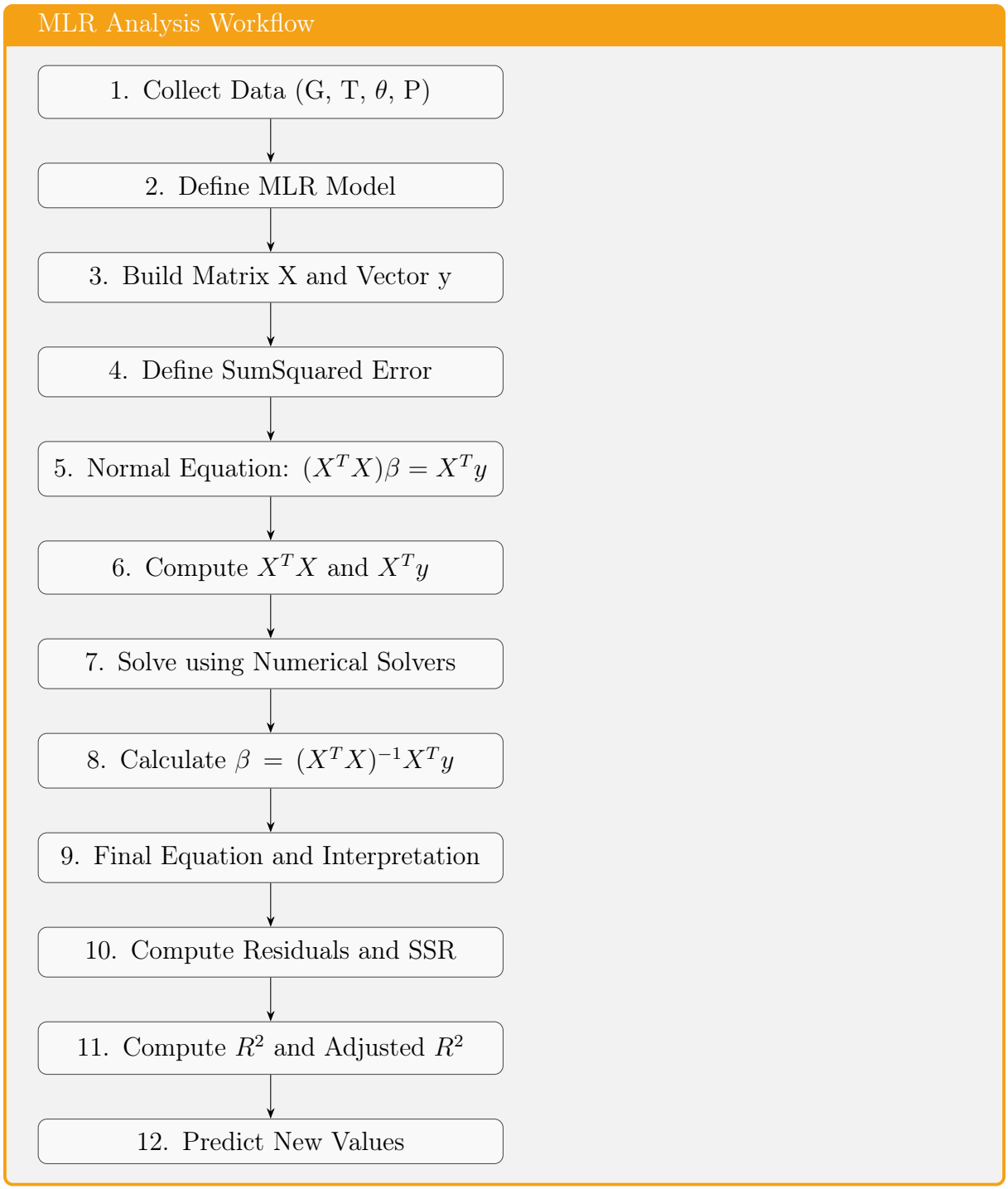


Figure 3.4: Refined workflow of the MLR Analysis.

3.2 Task Evaluation

Task 1: Max and Min Power

Since the model used is **linear**, it does not have any interior critical points because all partial derivatives are constant and never become zero. Therefore, any maximum or minimum value must occur at the **boundary of the input space**, and for a 3D rectangular domain, these extrema occur at the **corner points**.

To evaluate this, we generate all possible corner combinations of the input variables:

Corer Combinations

```
corners = list(product([200, 1000], [15, 45], [0, 60]))
```

We used `itertools.product([200, 1000], [15, 45], [0, 60])` to create all possible combinations of the input variables G, T, and θ. This generates every corner of the 3D input space by pairing each value of one variable with every value of the others, resulting in $2^3 = 8$ corner points.

The result is converted into a list so that we can easily iterate through each combination.

Next, we evaluate the trained linear regression model at each corner point:

Predictive Solution

```
powers = [lr.predict([[G, T, th]])[0] for G, T, th in corners]
```

Here, `lr.predict()` computes the predicted power output for each combination. Each input is passed as a 2D array `[[G, T, th]]` because scikit-learn models require 2D input even for a single sample.

The index `[0]` is used because the prediction function returns an array, and we only need the scalar predicted power value.

Finally, we determine the extreme values:

Max and Min

```
max_i = np.argmax(powers)
min_i = np.argmin(powers)
```

These functions return the indices of the maximum and minimum predicted power values, allowing us to identify the best-case and worst-case operating conditions.

Overall, this step evaluates the model at all extreme boundary points of the input space to analyze its behavior under the most favorable and least favorable input conditions.

Task 2 : Gradient

Task 2 Code Explanation

```
grad = np.array([bG, bT, bth])
unit = grad / np.linalg.norm(grad)
```

Dividing by the norm gives the unit vector pointing in the direction of steepest ascent.

We made the gradient vector by hand using the trained linear regression model's coefficients (`lr.coef_`), which are stored as b_G, b_T, and b_θ. Each coefficient shows how much power changes when one input variable changes:

- b_G is how sensitive it is to G.
- b_T is how sensitive it is to T.
- b_θ is how sensitive it is to θ.

We put them together into a NumPy array so that we can treat the gradient as a single mathematical vector for other operations, such as analyzing direction and finding dot products.

Task 3 : Steepest Descent

Task 3 Code Explanation

```
neg_grad = -grad
```

The direction of fastest power decrease is the negative gradient, corresponding to decreasing irradiance (cloud cover) and increasing the angle of incidence.

This step is used to figure out the best way to cut down on power.

Task 4 : Directional Derivative

Task 4 Code Explanation

```
d_unit = d / np.linalg.norm(d)
dir_deriv = np.dot(grad, d_unit)
```

We first use `np.linalg.norm(d)` to normalise the direction vector **d**. This changes **d** into a unit vector `d_unit`, meaning only direction matters and not size. Next, we use the dot product `np.dot(grad, d_unit)` to find the directional derivative. This shows how fast the power is changing in the direction of `d_unit`. The `dir_deriv` tells you how quickly power goes up or down in that direction.

Task 5 :Triple Integral

Task 5 Code Explanation

```
result, _ = integrate.tplquad(P, G1, G2, T1, T2, th1, th2)
avg_power = result / ((G2-G1) * (T2-T1) * (th2-th1))
```

`scipy.integrate.tplquad` computes the triple integral numerically. Dividing by the volume of the domain gives the average power over all operating conditions. The `d_unit` makes sure of fair direction comparison (size = 1) and `dir_deriv` determines the rate of change of power in that direction.

Task 6 : Location Performance

Task 6 Code Explanation

```
loc_powers = {name: lr.predict([vals])[0]
              for name, vals in locations.items()}
```

We built a dictionary using dict comprehension for evaluation of power at several predefined locations. `locations.items()` returns tuples consisting of `name` (location name) and `vals` (input values G, T, θ). For each location we supply `vals` to `lr.predict([vals])`. Wrapping with `[]` is necessary due to the two-dimensional requirement of the model. We take the output from the first index `[0]` because we receive an array.

4. Part II : Handwritten Solutions

4.1 Task 2.1: Handwritten Solution

This section presents the step-by-step handwritten analytical solutions corresponding to each task of the project. The solutions are based on the regression model derived from the dataset, expressed as follows.

$$P(G, T, \theta) = 0.808\,G - 2.548\,T - 4.9204\,\theta + 223.0904$$

We first included this because it covers all the points we addressed in our previous assignments; however, in order to reduce the effort of handwriting, we have also included it in a non-handwritten format.
Kindly refer to Chapter 5.

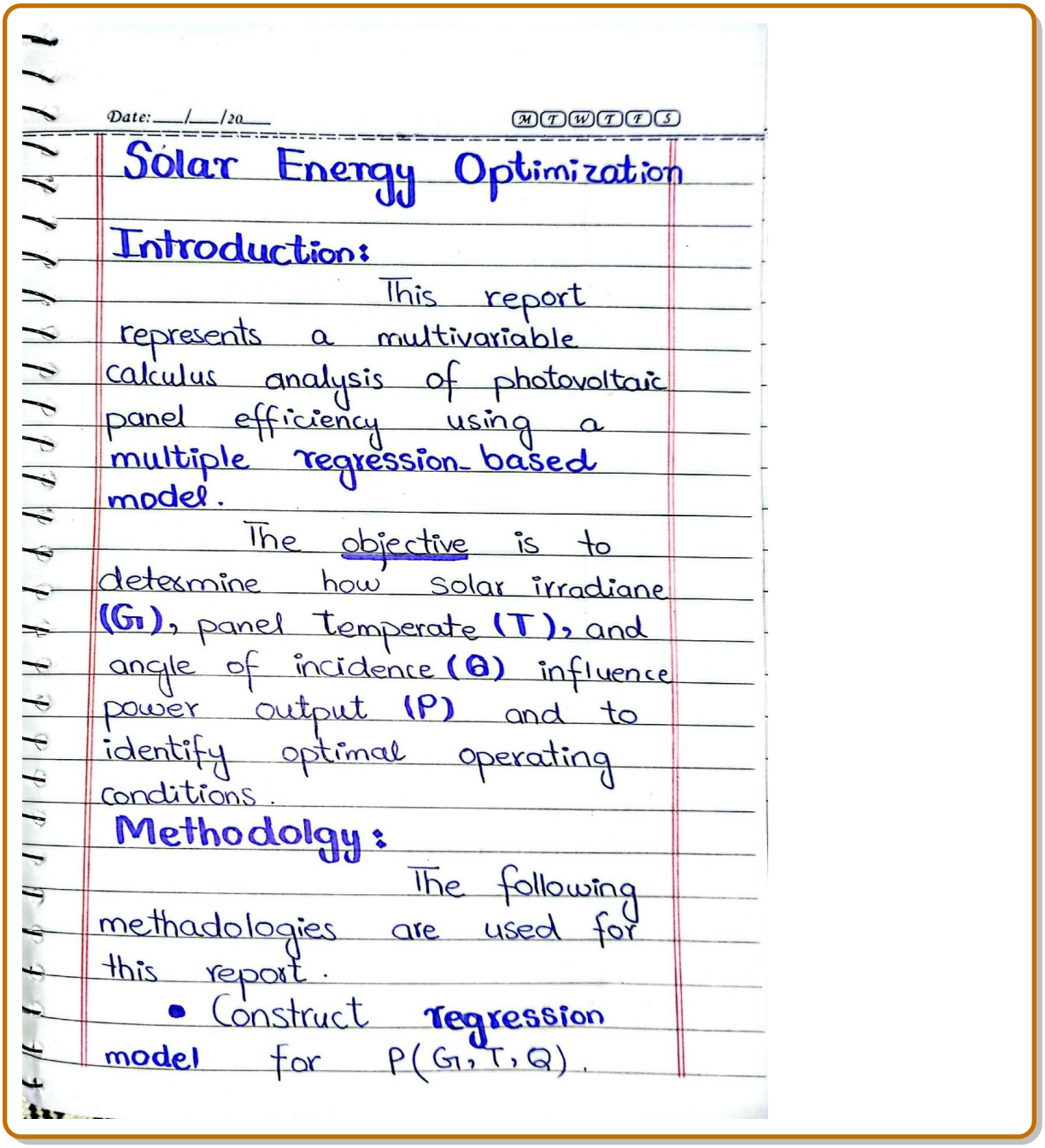
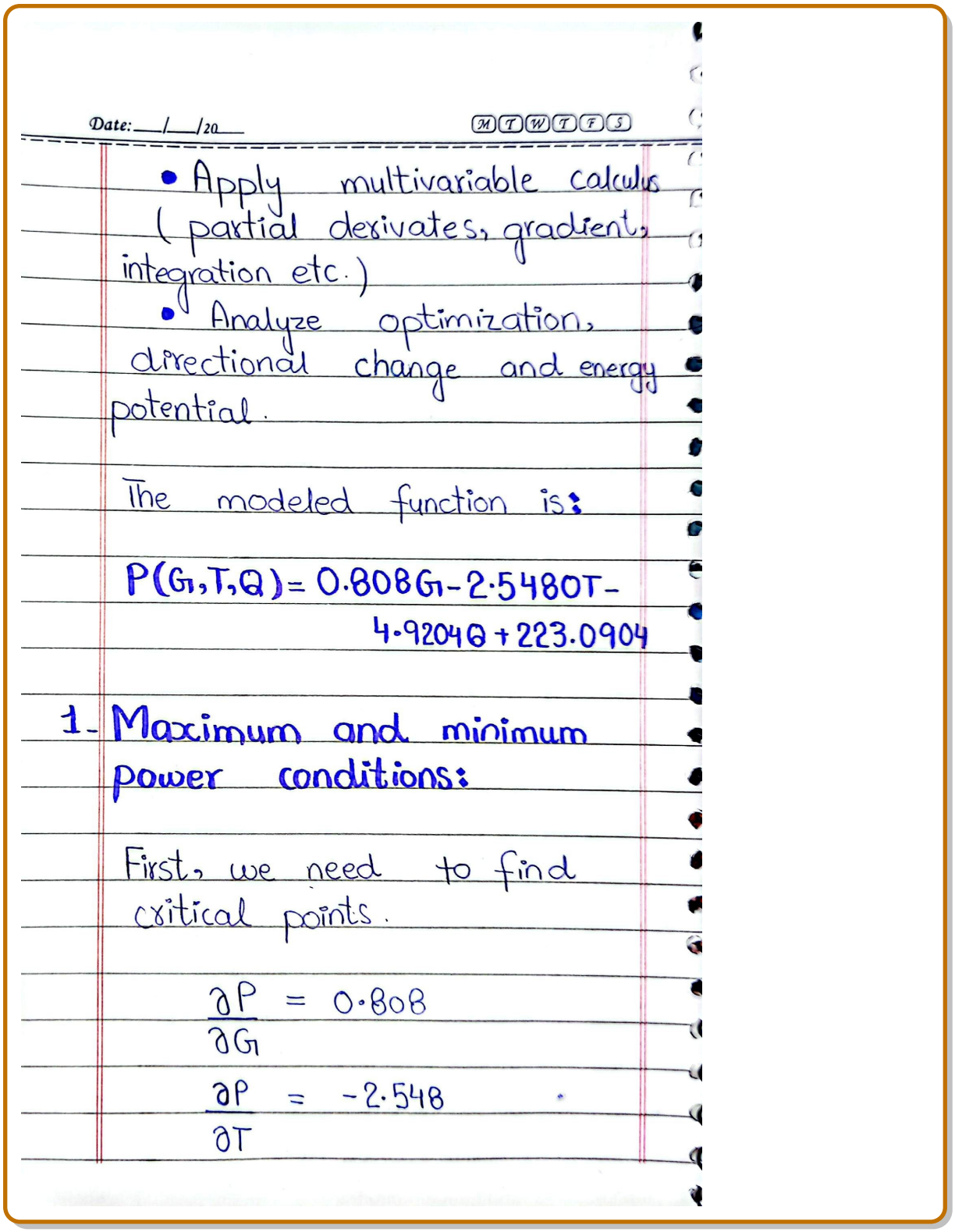


Figure 4.1: Handwritten Solution Page 1



Date: __/__/20__ M T W T F S

- Apply multivariable calculus (partial derivates, gradient, integration etc.)
- Analyze optimization, directional change and energy potential.

The modeled function is:

$$P(G,T,Q) = 0.808G - 2.548OT - 4.9204Q + 223.0904$$

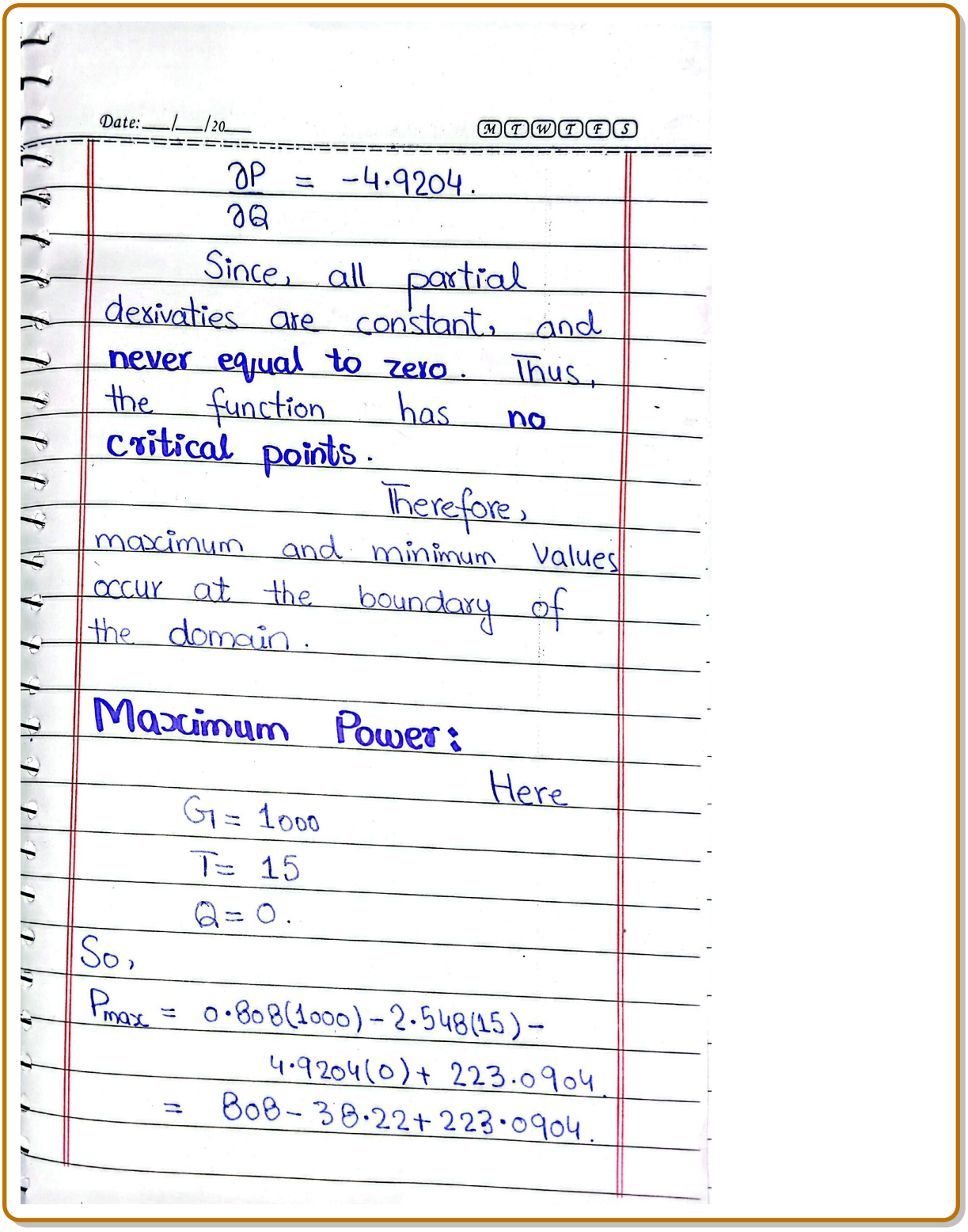
1- **Maximum and minimum power conditions:**

First, we need to find critical points.

$$\frac{\partial P}{\partial G} = 0.808$$

$$\frac{\partial P}{\partial T} = -2.548$$

Figure 4.2: Handwritten Solution Page 2



Date: __/__/20__ M T W T F S

$\frac{\partial P}{\partial Q} = -4.9204.$

Since, all partial dexivaties are constant, and **never equal to zero**. Thus, the function has **no critical points**.

Therefore, maximum and minimum values occur at the boundary of the domain.

Maximum Power:

Here

$G_1 = 1000$

$T = 15$

$Q = 0.$

So,

$P_{max} = 0.808(1000) - 2.548(15) - 4.9204(0) + 223.0904$

$= 808 - 38.22 + 223.0904.$

Figure 4.3: Handwritten Solution Page 3

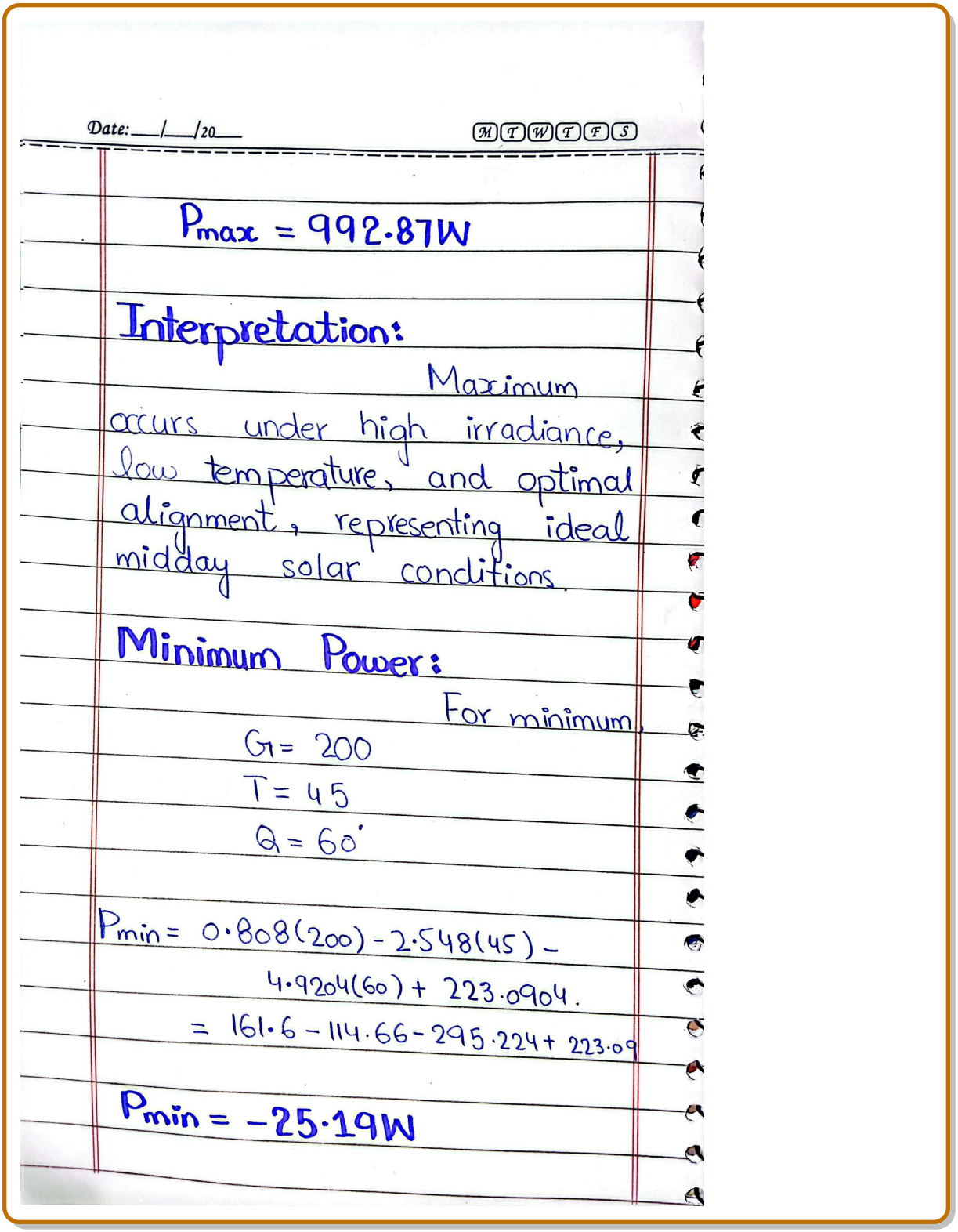
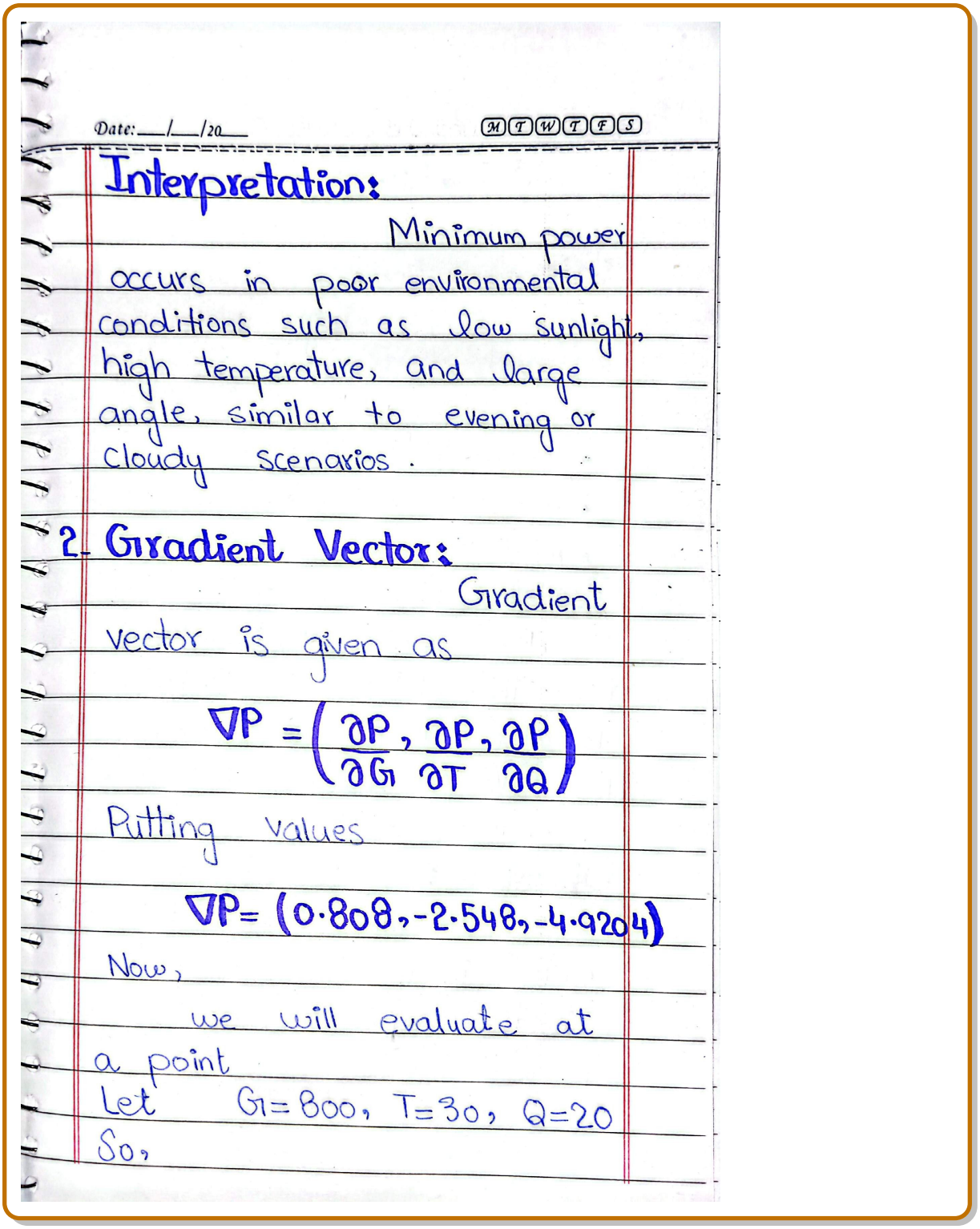


Figure 4.4: Handwritten Solution Page 4



Date: __/__/20__ M T W T F S

Interpretation:

Minimum power occurs in poor environmental conditions such as low sunlight, high temperature, and large angle, similar to evening or cloudy scenarios.

2. Gradient Vector:

Gradient vector is given as

$$\nabla P = \left(\frac{\partial P}{\partial G}, \frac{\partial P}{\partial T}, \frac{\partial P}{\partial Q}\right)$$

Putting values

$$\nabla P = (0.808, -2.548, -4.9204)$$

Now,

we will evaluate at a point

Let $G = 800$, $T = 30$, $Q = 20$

So,

Figure 4.5: Handwritten Solution Page 5

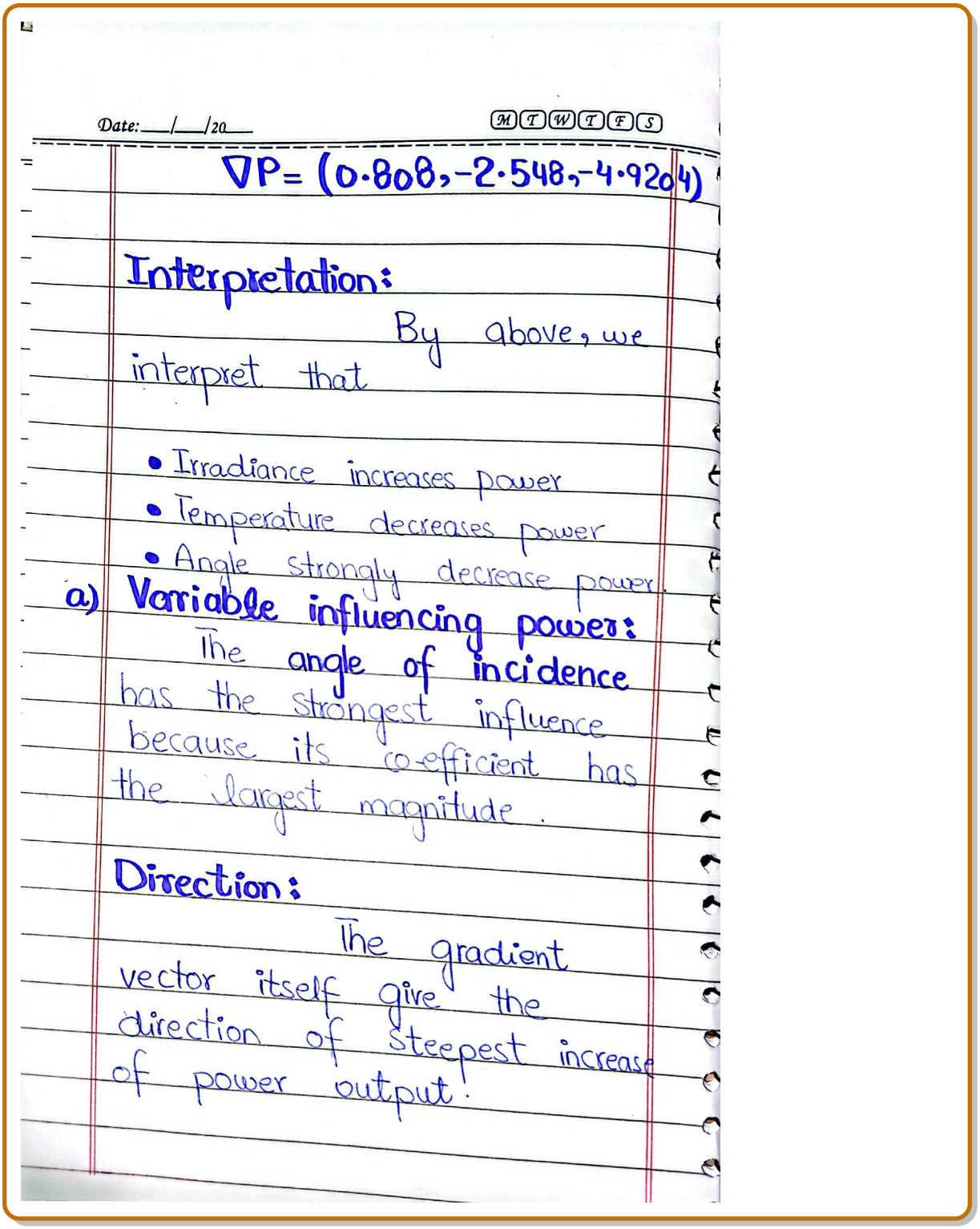


Figure 4.6: Handwritten Solution Page 6

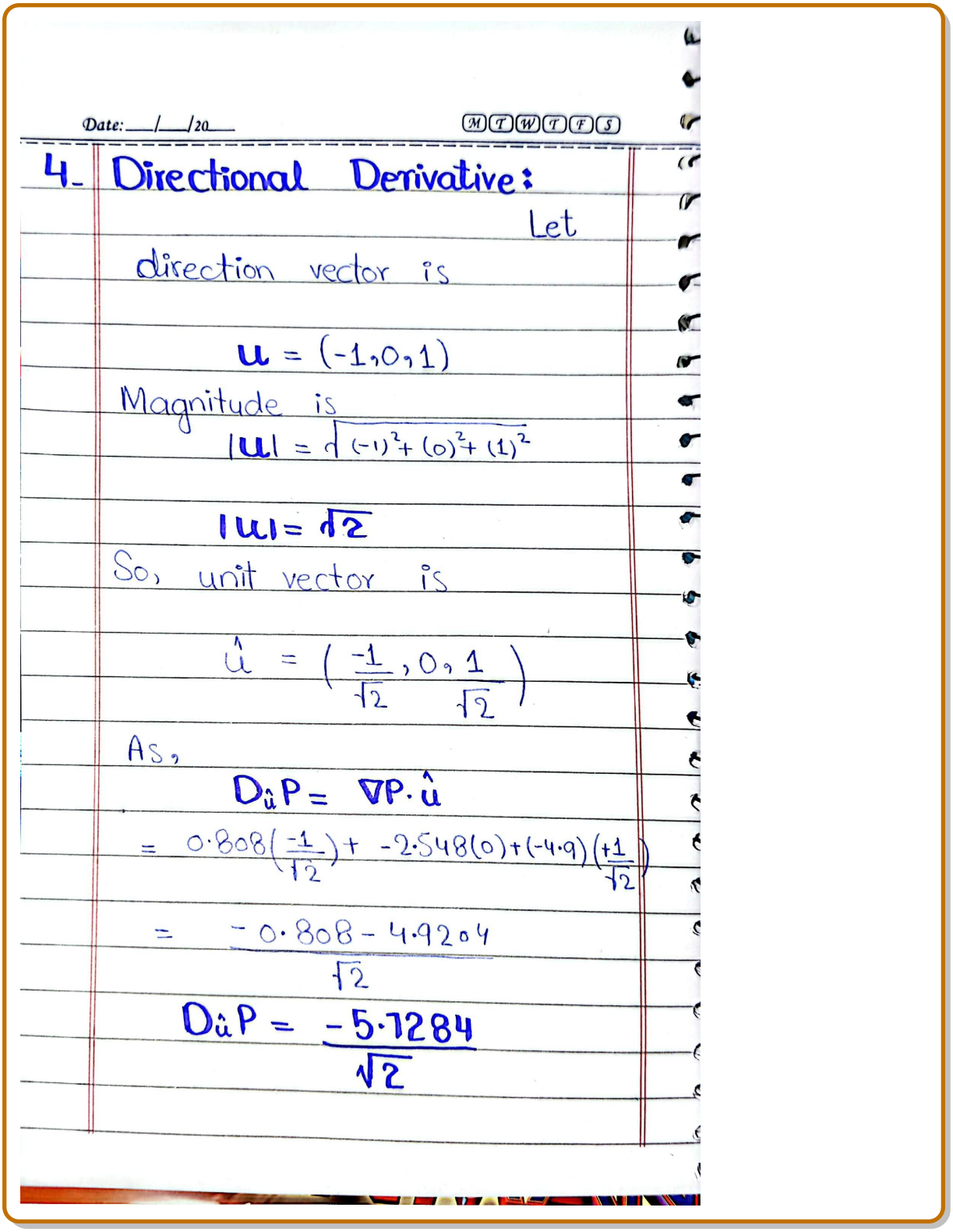


Figure 4.7: Handwritten Solution Page 7

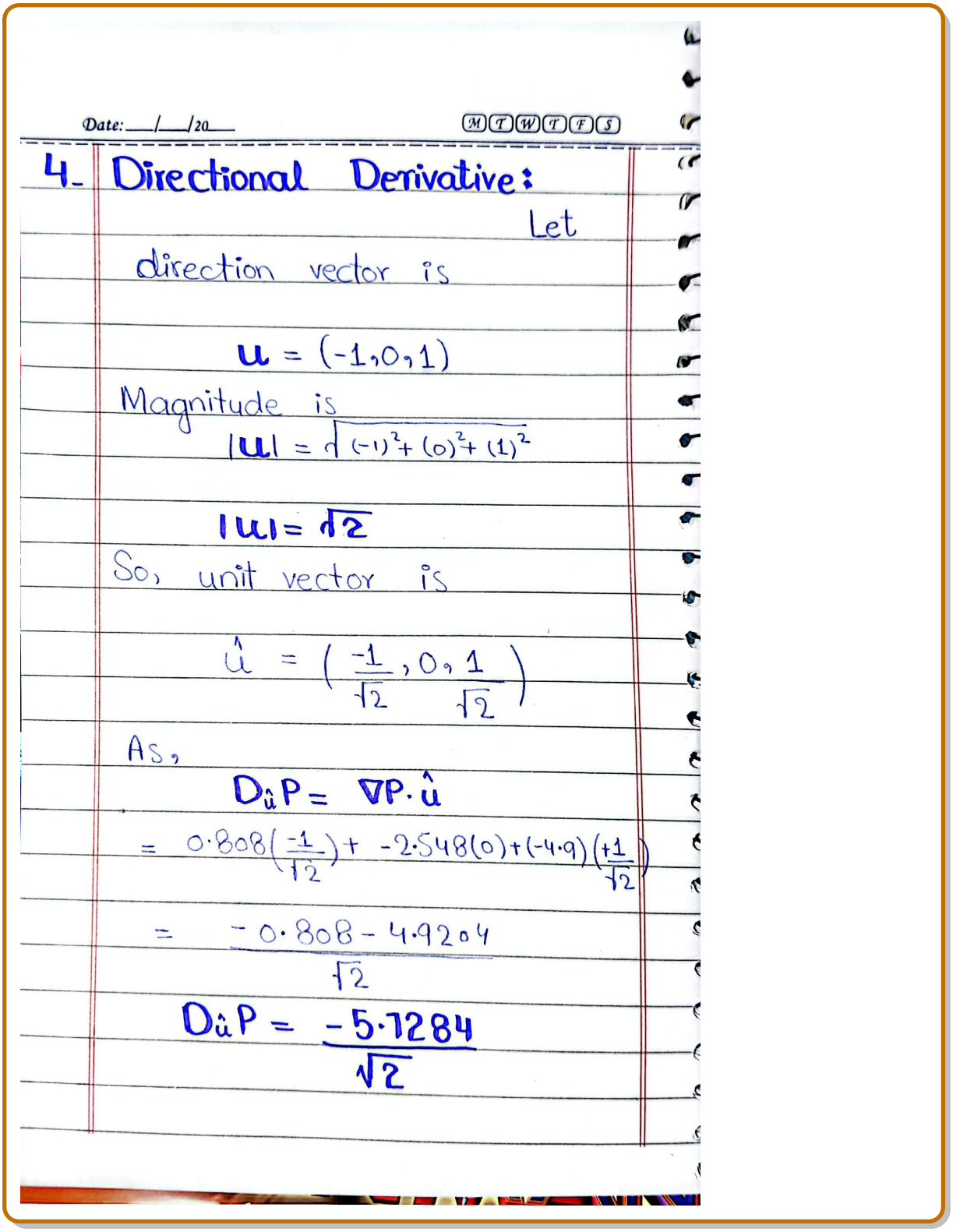


Figure 4.8: Handwritten Solution Page 8

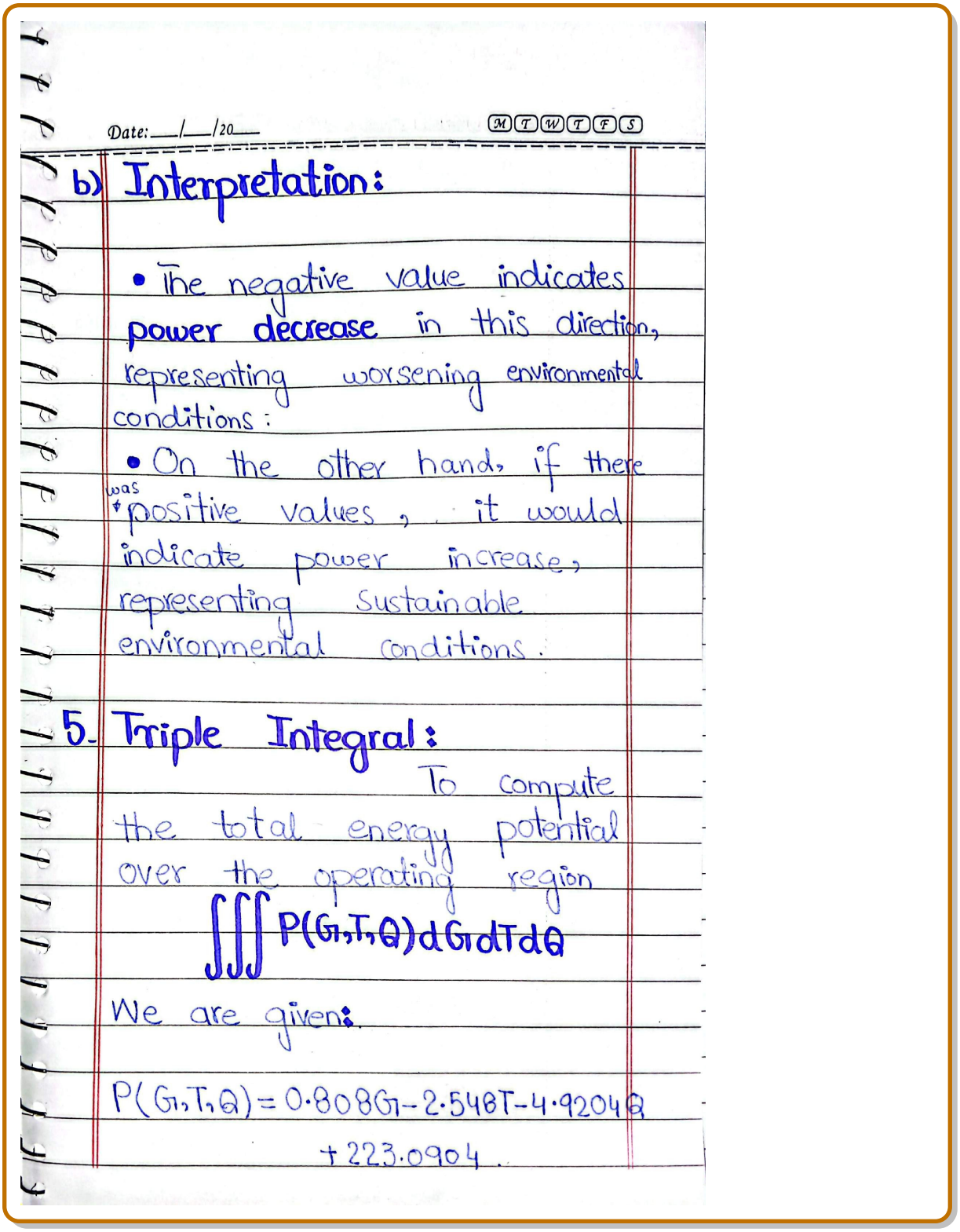


Figure 4.9: Handwritten Solution Page 9

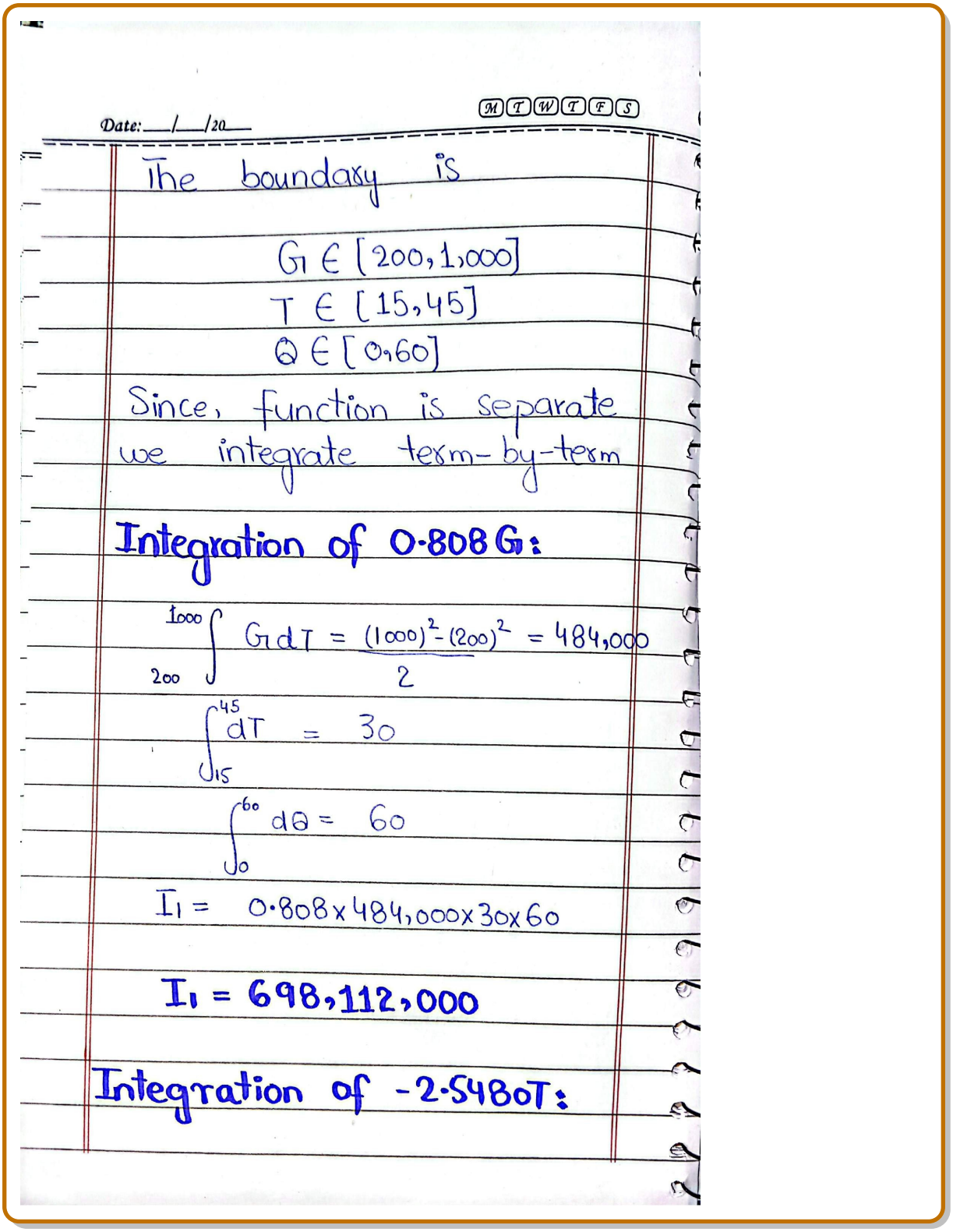


Figure 4.10: Handwritten Solution Page 10

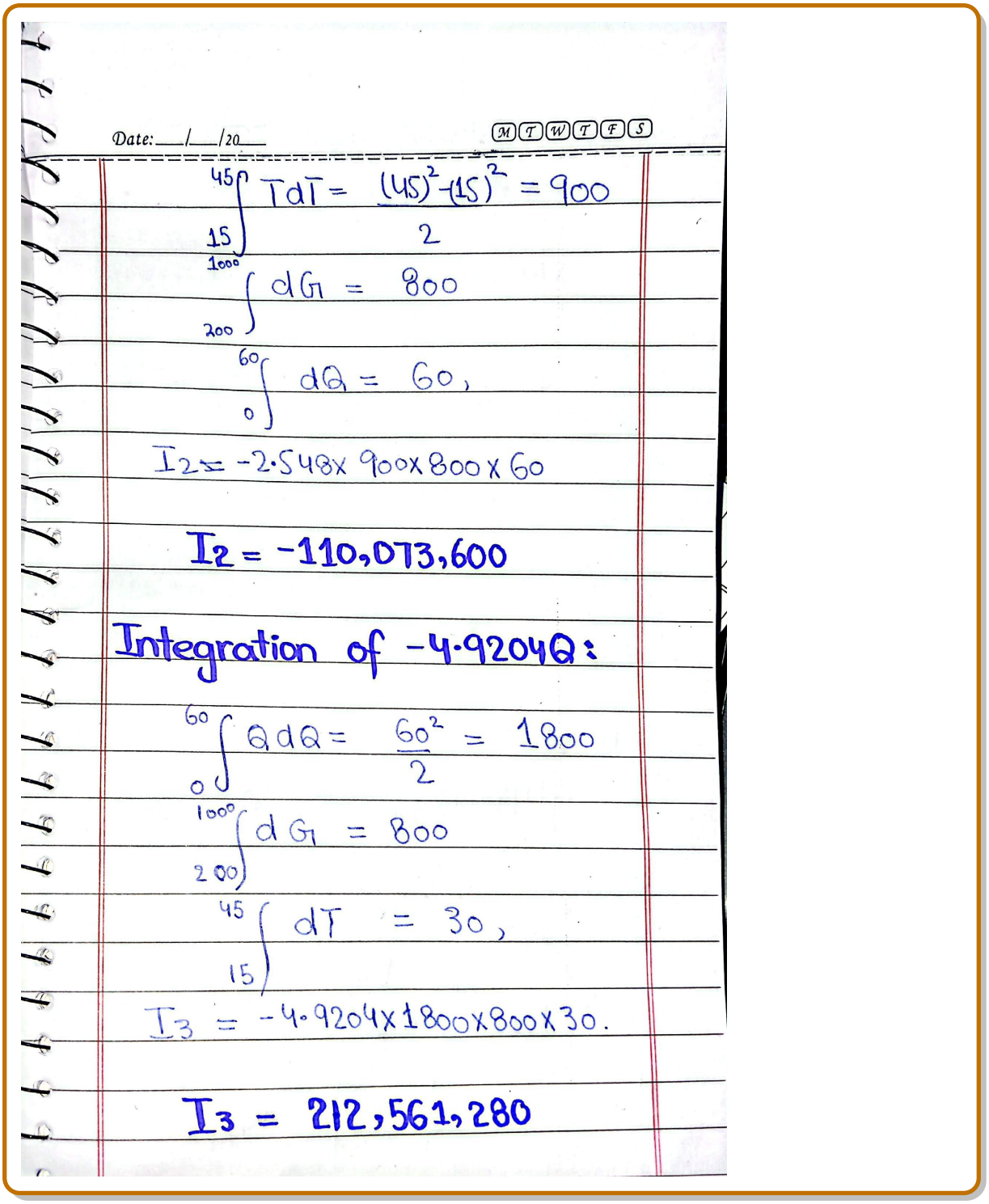
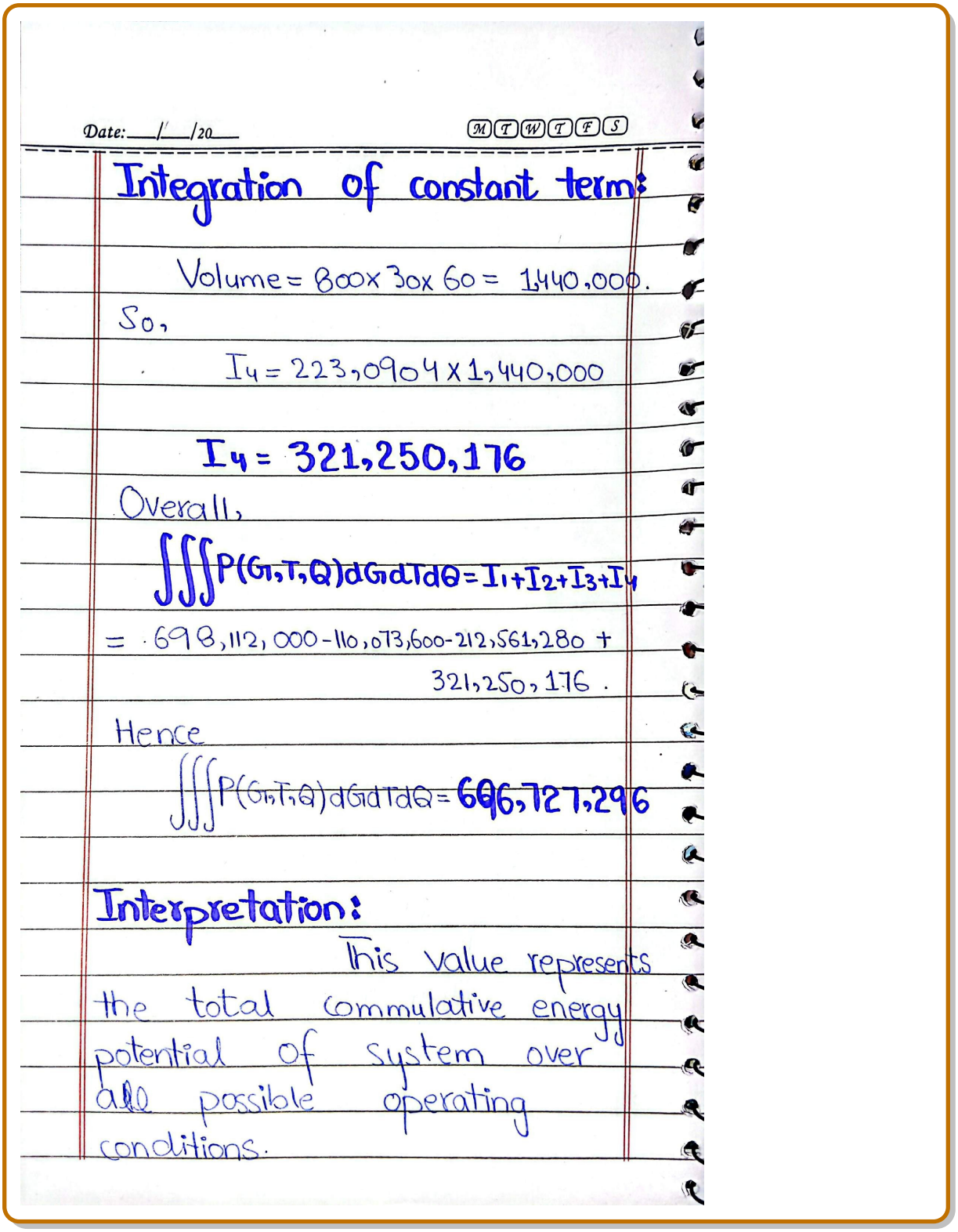


Figure 4.11: Handwritten Solution Page 11



Date: __/__/20__ M T W T F S

Integration of constant term:

Volume = 800 × 30 × 60 = 1,440,000.

So,

$I_4 = 223,0904 \times 1,440,000$

$I_4 = 321,250,176$

Overall,

$$\iiint P(G,T,\Theta)\,dG\,dT\,d\Theta = I_1 + I_2 + I_3 + I_4$$

$= 698,112,000 - 110,073,600 - 212,561,280 + 321,250,176.$

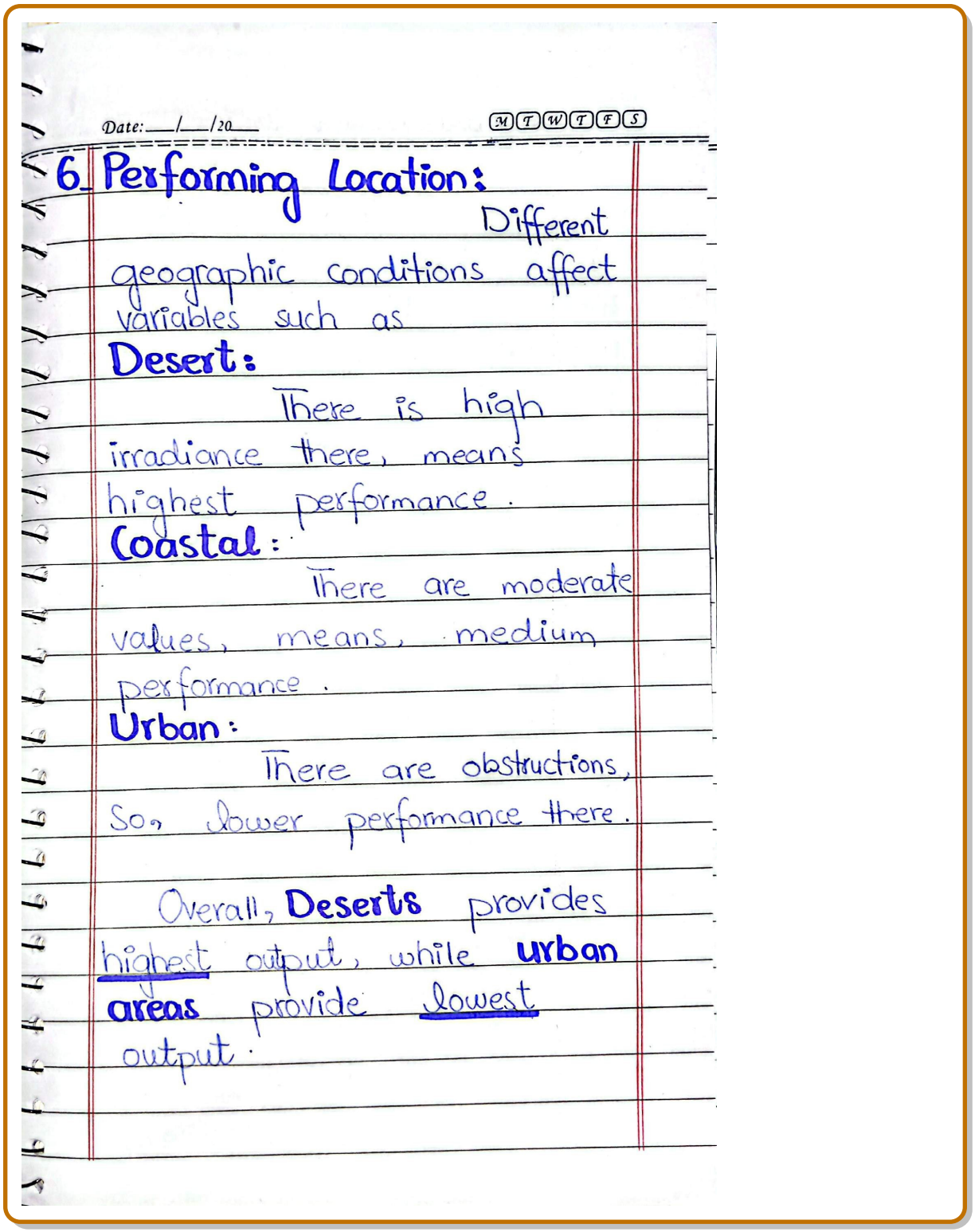
Hence

$$\iiint P(G,T,\Theta)\,dG\,dT\,d\Theta = 696,727,296$$

Interpretation:

This value represents the total commulative energy potential of system over all possible operating conditions.

Figure 4.12: Handwritten Solution Page 12



Date: __/__/20__ M T W T F S

6. Performing Location:

Different geographic conditions affect variables such as

Desert:

There is high irradiance there, means highest performance.

Coastal:

There are moderate values, means, medium performance.

Urban:

There are obstructions, So, lower performance there.

Overall, **Deserts** provides highest output, while **urban areas** provide lowest output.

Figure 4.13: Handwritten Solution Page 13

5. Docx Written Solution Covering Tasks 1 to 6

This section presents the step-by-step analytical (by-hand) solutions corresponding to each task. All solutions are based on the regression model. This section was included to fulfill the requirement that all LaTeX content must be typed.

$$P(G,T,\theta) \;=\; 0.808\,G \;-\; 2.548\,T \;-\; 4.9204\,\theta \;+\; 223.0904$$

5.1 Task 1: Maximum and Minimum Power Conditions

To find the maximum and minimum power output, critical points are first identified by computing the partial derivatives of the modelled function.

$$\frac{\partial P}{\partial G} = 0.808, \qquad \frac{\partial P}{\partial T} = -2.548, \qquad \frac{\partial P}{\partial \theta} = -4.9204$$

Since all partial derivatives are constants and never equal to zero, the function has no critical points. Therefore, maximum and minimum values occur at the boundaries of the domain.

Maximum Power: The boundary values that maximize P are $G = 1000$ W/m^2, $T = 15\,^\circ$C, and $\theta = 0^\circ$.

$$\begin{aligned} P_{\max} &= 0.808(1000) - 2.548(15) - 4.9204(0) + 223.0904 \\ &= 808 - 38.22 - 0 + 223.0904 \\ &= \boxed{992.87 \text{ W}} \end{aligned}$$

Interpretation: Maximum power occurs under high irradiance, low temperature, and optimal panel alignment, representing ideal midday solar conditions.

Minimum Power: The boundary values that minimize P are $G = 200$ W/m^2, $T = 45\,^\circ$C, and $\theta = 60^\circ$.

$$\begin{aligned} P_{\min} &= 0.808(200) - 2.548(45) - 4.9204(60) + 223.0904 \\ &= 161.6 - 114.66 - 295.224 + 223.0904 \\ &= \boxed{-25.19 \text{ W}} \end{aligned}$$

Interpretation: Minimum power occurs under poor environmental conditions such as low sunlight, extreme heat, and a large angle of incidence, similar to evening or heavily cloudy scenarios.

5.2 Task 2: Gradient Vector $\nabla P(G, T, \theta)$

For a linear function, the gradient vector is simply the vector of partial derivative coefficients. The gradient is given by:

$$\nabla P = \left(\frac{\partial P}{\partial G}, \frac{\partial P}{\partial T}, \frac{\partial P}{\partial \theta} \right) = (0.808,\ -2.548,\ -4.9204)$$

Evaluated at the representative midday operating point $G = 800\ \text{W/m}^2$, $T = 30\,^\circ\text{C}$, $\theta = 20^\circ$:

$$\nabla P = (0.808,\ -2.548,\ -4.9204)$$

(a) Variable Influencing Power the Most: The angle of incidence (θ) has the strongest influence on power output at this operating point, because its partial derivative coefficient $|-4.9204|$ has the largest magnitude among the three variables.

(b) Direction of Fastest Increase: The gradient vector itself gives the direction of steepest increase of power output. To increase power most rapidly, irradiance should increase, while temperature and angle of incidence should decrease.

5.3 Task 3: Steepest Descent

The direction of fastest power decrease is given by the negative of the gradient vector:

$$-\nabla P = (-0.808,\ 2.548,\ 4.9204)$$

Interpretation: Power decreases most rapidly in the direction where irradiance decreases, temperature increases, and angle of incidence increases. These conditions correspond to real-world scenarios such as passing cloud cover reducing irradiance, heat build-up during the afternoon, or the panel angle increasing as the sun sets toward the horizon — all characteristic of sunset or overcast conditions.

5.4 Task 4: Directional Derivative

(a) A custom direction vector is chosen to represent the weather scenario where irradiance is dropping while the angle of incidence is increasing and temperature remains unchanged:

$$\mathbf{u} = (-1,\ 0,\ 1), \qquad |\mathbf{u}| = \sqrt{(-1)^2 + 0^2 + 1^2} = \sqrt{2}, \qquad \hat{\mathbf{u}} = \left(\frac{-1}{\sqrt{2}},\ 0,\ \frac{1}{\sqrt{2}}\right)$$

The directional derivative is computed as:

$$\begin{aligned} D_{\hat{u}}P &= \nabla P \cdot \hat{\mathbf{u}} \\ &= 0.808\left(\frac{-1}{\sqrt{2}}\right) + (-2.548)(0) + (-4.9204)\left(\frac{1}{\sqrt{2}}\right) \\ &= \frac{-0.808 - 4.9204}{\sqrt{2}} \\ &= \frac{-5.7284}{\sqrt{2}} \\ &\approx \boxed{-4.049 \text{ W per unit change}} \end{aligned}$$

(b) Interpretation: The negative directional derivative value (-4.049) indicates that power output decreases in this direction, representing worsening environmental conditions. For energy grid management, a negative directional derivative signals that battery storage should begin dispatching power to compensate for the anticipated drop in solar generation. A positive directional derivative, conversely, would indicate improving conditions where continued solar generation is sufficient to meet demand without battery dispatch.

5.5 Task 5: Compute Triple Integral Over a Region

The triple integral of $P(G, T, \theta)$ is computed over the realistic daily operating envelope $G \in [200, 1000]$, $T \in [15, 45]$, $\theta \in [0, 60]$:

$$\iiint P(G, T, \theta)\, dG\, dT\, d\theta$$

Since the function is separable (linear), the integral is computed term by term.

Integration of $0.808\, G$ term:

$$\int_{200}^{1000} G\, dG = \frac{1000^2 - 200^2}{2} = 484{,}000 \qquad I_1 = 0.808 \times 484{,}000 \times 30 \times 60 = 698{,}112{,}000$$

Integration of $-2.548\, T$ term:

$$\int_{15}^{45} T\, dT = \frac{45^2 - 15^2}{2} = 900 \qquad I_2 = -2.548 \times 900 \times 800 \times 60 = -110{,}073{,}600$$

Integration of $-4.9204\, \theta$ term:

$$\int_{0}^{60} \theta\, d\theta = \frac{60^2}{2} = 1{,}800 \qquad I_3 = -4.9204 \times 1{,}800 \times 800 \times 30 = -212{,}561{,}280$$

Integration of constant 223.0904 term:

$$\text{Volume} = 800 \times 30 \times 60 = 1{,}440{,}000 \qquad I_4 = 223.0904 \times 1{,}440{,}000 = 321{,}250{,}176$$

Total Triple Integral:

$$\iiint P\, dG\, dT\, d\theta = 698{,}112{,}000 - 110{,}073{,}600 - 212{,}561{,}280 + 321{,}250{,}176 = \boxed{696{,}727{,}296}$$

Interpretation: This value represents the total cumulative energy potential of the solar panel system over all possible operating conditions within the defined domain. It provides an estimate of the total energy generation capacity across the three-dimensional operational space.

5.6 Task 6: Performing Location

Different geographic conditions affect the input variables G, T, and θ. Representative values are assigned per location category and the power output is computed using the regression model.

Location	G (W/m^2)	T (°C)	θ (°)	P (W)
Desert	950	40	10	839.57
High-Altitude	900	15	20	813.66
Coastal	700	28	25	594.34
Urban	500	35	35	365.70

Table 5.1: Table 1: Predicted Power Output by Geographic Location

Ranking: Desert > High-Altitude > Coastal > Urban

Highest Performing : Desert (839.57 W): Desert regions receive the most intense and unobstructed solar irradiance ($G \approx 950$ W/m^2). Despite elevated temperatures, the dominant effect of high irradiance results in the maximum predicted power output of 839.57 W.

Second : High-Altitude (813.66 W): High-altitude locations benefit from very low temperatures ($T \approx 15\,^\circ$C) which significantly reduce thermal losses, compensating for slightly lower irradiance. Predicted output is 813.66 W.

Third : Coastal (594.34 W): Coastal environments exhibit moderate conditions across all variables, yielding a medium-range predicted output of 594.34 W. Moderate humidity and cloud cover limit peak irradiance.

Lowest Performing : Urban (365.70 W): Urban areas suffer from multiple simultaneous losses: reduced irradiance due to shading and atmospheric pollution ($G \approx 500$ W/m^2), elevated temperatures from the urban heat island effect ($T \approx 35\,^\circ$C), and poor panel alignment due to building constraints ($\theta \approx 35^\circ$). All three factors combine to produce the minimum predicted output of 365.70 W.

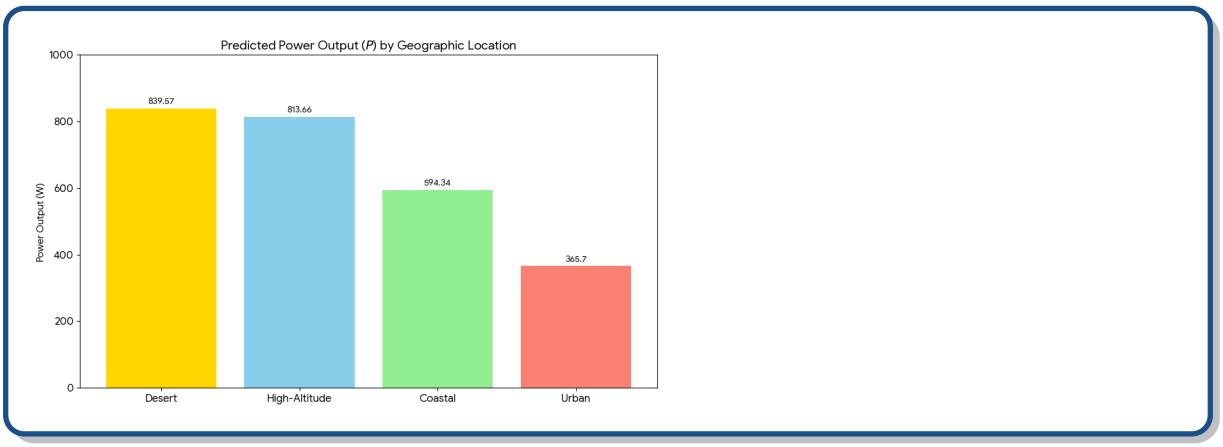


Figure 5.1: Python generated graph showing the relationship of power and location

6. Analysis of Results and Conclusion

6.1 Analysis of Results

By comparing our manual calculations with the Python implementation, we come to the conclusion that both approaches are fully consistent and validate each other.

- ✔ From the corner evaluation done in Task 1, we observed that the maximum power is 992.87 W at ($G = 1000$, $T = 15$, $\theta = 0$) and the minimum power is -25.19 W at ($G = 200$, $T = 45$, $\theta = 60$). By matching our handwritten solutions with our Python code using `itertools.product` and `lr.predict()`, we come to the conclusion that the function has no internal critical points and the extrema occur only at the boundary values of the input space.
- ✔ From Task 2, we computed the gradient vector as $(0.8080, -2.5480, -4.9204)$ using the coefficients from `lr.coef_`. By matching our handwritten solutions with our code, we come to the conclusion that irradiance positively affects power, while temperature and angle of incidence negatively affect it, with angle having the strongest influence due to the largest magnitude coefficient.
- ✔ In Task 3, we calculated the negative gradient as $-\nabla P = (-0.8080, 2.5480, 4.9204)$. By matching our handwritten solutions with our Python operation `neg_grad = -grad`, we come to the conclusion that this direction represents the fastest decrease in power, corresponding to unfavourable environmental conditions such as low sunlight and high temperature.
- ✔ In Task 4, we computed the directional derivative using the dot product between the gradient and a unit direction vector. By matching our handwritten solutions with our code using `np.dot(grad, d_unit)`, we come to the conclusion that a negative directional derivative indicates a decrease in power along that direction.
- ✔ For Task 5, we performed triple integration using `scipy.integrate.tplquad` to calculate total power over the full domain. By matching our handwritten solutions with our Python implementation, we come to the conclusion that this represents the total energy potential across all operating conditions, and the average power is obtained by dividing by the volume of the domain.

6.2 Conclusion

1. By matching all our analytical results with the Python implementation, we come to the conclusion that the model behaves in a completely linear and predictable manner, since all partial derivatives are constant. This means the system does not have any nonlinear optimization behavior, and performance changes occur uniformly across the input space.

2. We also observe that the angle of incidence (θ) is the most sensitive variable in the system. By comparing coefficient magnitudes and directional effects in both the gradient and directional derivative, we come to the conclusion that even small changes in angle have a stronger negative impact on power compared to changes in temperature or irradiance. This indicates that physical alignment of solar panels is more critical than environmental fluctuations in this model.

3. Another important observation is that temperature consistently reduces output, confirming a strong inverse relationship between heat and efficiency. By matching this with both gradient analysis and corner evaluations, we come to the conclusion that high-temperature conditions significantly degrade photovoltaic performance even when irradiance is high.

4. From the location-based evaluation, we further conclude that environmental context plays a major role in overall system efficiency. Desert regions consistently show the highest predicted output due to high irradiance and lower structural interference, while urban environments show reduced performance due to obstructions and suboptimal exposure.

5. Finally, by combining integration results with boundary analysis, we come to the conclusion that while local performance varies, the overall energy potential of the system is primarily determined by operating within optimal boundary conditions rather than average internal behavior.

6.3 What We Learned from This Project

- By matching our mathematical analysis with the Python implementation, we come to the conclusion that photovoltaic power output is effectively modeled using a multivariable linear regression approach, which forms the main foundation of this project.
- Through self-study and analysis of the regression model, which was the best part as new learners, when we read the modeling of function then we understand: whenever we have a sample space having no particular formulae, how can we model the function that predicts the closest value? For that we use multi linear regression. We want to include key articles that helped us in understanding how these functions can be found out. As a programmer and learner it is our duty to know what that particular line did, and by understanding that we have written our code.
- We also come to the conclusion that the system behaves in a linear and predictable manner since all partial derivatives are constant. Because of this, maximum and minimum power values occur at the boundary conditions rather than at any internal critical points.
- Overall, by combining regression modeling with calculus-based tools in Python such as gradients and directional derivatives, we learned how to analyze and optimize real-world solar energy systems. The most important insight from this project is that proper panel alignment and temperature control are more critical than irradiance variations in improving efficiency.

Team Contributions

Member	Tasks Performed	Share
Ahmad Ali (25i-0817) Group Leader	Document formation, Python solution, Brainstorming in modeling of function	70% / 1% / 33.3%
Liaba Naveed 25i-0818	Handwritten solution, Brainstorming in modeling of function	100% / 33.3%
Muhammad Osaid Zahid 25i-0524	Python code, Brainstorming in modeling of function	99% / 33.3%

References

For Multi Linear Regression

[1] Multiple Linear Regression *Investopedia*
https://www.investopedia.com/terms/m/mlr.asp

[2] Multiple Linear Regression Guide *Scribbr*
https://www.scribbr.com/statistics/multiple-linear-regression/

Coding Platform

Visual Studio Code with Python 3.14.4

Flowcharts

Draw.io (diagrams.net)

Annexure: Additional Code Snippets

Code for Compute Triple Integral Over a Region

```
#Integrate P(G,T,) over full operating envelope
G1, G2   = 200, 1000
T1, T2   = 15,  45
th1, th2 = 0,   60

def P(th, T, G):
    return b0 + bG*G + bT*T + b*th

result, _ = integrate.tplquad(P, G1, G2, T1, T2, th1, th2)
avg_power = result / ((G2-G1) * (T2-T1) * (th2-th1))

print("\nTASK 5: TRIPLE INTEGRAL")
print(f" P dG dT d = {result:.2f}")
print(f"Average power over operating envelope = {avg_power:.2f} W")
```

Code for Compute the Gradient Vector at a Given Point

```
G0  = df['G_Irradiance'].mean()
T0  = df['Temperature'].mean()
th0 = df['Theta_deg'].mean()
grad = np.array([bG, bT, bth])

print("\nTASK 2: GRADIENT")
print(f"Operating point: G={G0:.1f}, T={T0:.1f}, theta={th0:.1f}")
print(f"Gradient P = [{bG:.4f}, {bT:.4f}, {bth:.4f}]")
print("Largest component:", names[np.argmax(np.abs(grad))])

unit = grad / np.linalg.norm(grad)
print(f"Fastest increase direction (unit vector): "
      f"[{unit[0]:.4f}, {unit[1]:.4f}, {unit[2]:.4f}]")
```

These were the demanded tasks. All tasks were solved both by hand and by compiler, then compared and conclusions were drawn.


‘BEST MVC PROJECT OF ALL TIME’
we learend a lot thanks